Scenarios

Scenario 1	
Demo 1 -> f1.txt to f4.txt	
git status;	- Not a Git Repository
git init;	- Initialized emty Git Preo
git status;	- It shows on which branch we are currently (though its totally logical, it's half truth - no brach is created yet) Currently it shows 2 things : 1. No commits yet 2. List of Untracked files
git add f1.txt f2.txt f3.txt;	- This msg adds the following file into the index(Staging area)
git status;	- Now it shows : 1. List of files to be committed and 2. untracked files : f4.txt
git commit -m "first commit";	- Files are committed locally, now physicial/actual main branch is created.
git status	- As 1 file is untracked so : it gives f4.txt
git restore --source=hash1 .	- No changes is done, so nothing is restored, everything stay as it is
check WD	
git status	- Same as it shows above n one of the steps : 1. untracked files : f4.txt
Scenario 2	
Demo 1 -> f1.txt to f4.txt	
git init;	
git status;	
git add f1.txt f2.txt f3.txt;	
git status	
git commit -m "first commit";	
git status	
update f2	- Change the text of f2.text
git status	- Now it shows 2 things : 1. Changes not staged for commit: f2.txt 2. Untracked files: f4.txt
git restore --source=hash1 .	- It restores the f2.txt from the last commit
check WD	
git status	- Same as it shows above n one of the steps : 1. untracked files : f4.txt
Scenario 3	
Demo 1 -> f1.txt to f4.txt	
git init;	
git status;	
git add f1.txt f2.txt f3.txt;	
git status;	

git commit -m "first commit"; git status	
update f2.txt	- Change the text of f2.text
git status	- Now it shows 2 things : 1. Changes not staged for commit: f2.txt 2. Untracked files: f4.txt
git add f2.txt	- It add f2.txt into the staging area.
git status	- Now it shows 2 things : 1. Changes to be committed: f2.txt a. modified: f2.txt 2. Untracked files: f4.txt
git restore --source=hash1 .	- It restores the f2.txt file from the last commit and puts it in the WD, it doesn't make any change to the same file in the Index Area.
check WD	- F2.txt is in both Changes to be commited: (Stagging Aread)and Changes not staged for commit: (WD)
git status	- Now it shows 3 different things : 1. Changes to be committed: modified: f2.txt 2. Changes not staged for commit: modified: f2.txt 3. Untracked files: f4.txt
Scenario 4	
Demo 1 -> f1.txt to f4.txt git init; git status; git add f1.txt f2.txt f3.txt; git status; git commit -m "first commit"; git status	
update f2, rm f3	- Updated the f2.txt file and remove the f3.txt file
git status	- Now it shows 2 different things : 1. Changes not staged for commit: modified: f2.txt deleted: f3.txt 2. Untracked files: f4.txt
git restore --source=hash1 .	- It restores the f3.txt file from the last commit
check WD	- f3.txt file is back and old f2.txt is back from last commit
git status	- Same as it shows above n one of the steps : 1. untracked files : f4.txt
Scenario 5	
Demo 1 -> f1.txt to f4.txt git init; git status; git add f1.txt f2.txt f3.txt; git status;	

git commit -m "first commit"; git status update f2, rm f3; git status	- Now it shows2 different things: 1. Changes not staged for commit: modified f2.txt, deleted f3.txt 2. Untracked files: f4.txt
git add f2.txt;	- It adds f2.txt into the staging area
git status	- Now it shows 3 different things : 1. Changes to be committed: modified: f2.txt 2. Changes not staged for commit: deleted: f3.txt 3. Untracked files: f4.txt
git restore --source=hash1 .	- It restores f2.txt and f3.txt from the last commit into the WD; it doesn't touch the Index Area, and f3.txt comes back.
check WD	- F2.txt is now old (matches last commit) in WD but Index still has the modified version staged; f3.txt is back and matches both Index and WD.
git status	- Now it shows 3 different things : 1. Changes to be committed: modified: f2.txt 2. Changes not staged for commit: modified: f2.txt 3. Untracked files: f4.txt
Scenario 6	
Demo 1 -> f1.txt to f4.txt git init; git status; git add f1.txt f2.txt f3.txt; git status; git commit -m "first commit"; git status update f2, rm f3	
git status	- It moves deleted f3.txt into the staging area
git add f3.txt	- Now it shows 3 different things :
git status	1. Changes to be committed: deleted: f3.txt 2. Changes not staged for commit: modified: f2.txt 3. Untracked files: f4.txt
git restore --source=hash1 . check WD	- It restores the deleted f3.txt file into the WD

git status	- f3.txt file is created as new, bcz the file is being tracked in index area, which shows tells that it was deleted, if it was nto tracked, then it will not be shown as new - Now it shows 3 different things : 1. Changes to be committed: modified: f2.txt 2. Untracked files: f3.txt, f4.txt
Scenario 7	
Demo 1 -> f1.txt to f4.txt git init; git status; git add f1.txt f2.txt f3.txt; git status git commit -m "first commit"; git status; update f2, rm f3 git status	
git add f2.txt f3.txt	- It moves the f2.txt in the staging area as it is changed and also add f3.txt in the staging area as it is deleted
git status	- Now it shows2 different things: 1. Changes not staged for commit: modified f2.txt, deleted f3.txt 2. Untracked files: f4.txt
git restore --source=hash1 .	- Now it restores f3.txt as new file as it is being tracked, and it also restores the f2.txt file from last commit without touching it in the index area
check WD git status	- Now it shows 3 different things : 1. Changes to be committed: deleted: f3.txt modified : f2.txt 2. Changes not staged for commit: modified: f2.txt 3. Untracked files: f3.txt, f4.txt
Scenario 8	
Demo 1 -> f1.txt to f4.txt git init; git status; git add f1.txt f2.txt f3.txt; git status; git commit -m "first commit"; git status;	
rm f3.txt; git status;	- It removes the f3.txt file from the WD - Now it shows 2 things :

	1. Changes not staged for commit: deleted: f3.txt 2. Untracked files: f4.txt
git add f3.txt; git status; git commit -m "second commit"; git status; git restore --source=hash1 . check WD git status git restore --source=hash2 . check WD git status	- It adds f3.txt file into the stagging area. - Now instead of f3.txt file being in Changes not staged for commit, it goes to Changes to be committed. - It creates another commit, in which we don't have f3.txt file - It only shows Untracked file : f4.txt - It restores the f3.txt file from the "first commit", as we have used 1st commit hash - f3.txt file is there as new one - Now it shows 1 things : 1. Untracked files: f3.txt, f4.txt - No changes as f3.txt is not tracked yet… - No changes in the WD - Same as above : 1. Untracked files: f3.txt, f4.txt
Scenario 9	
Demo 1 -> f1.txt to f4.txt git init; git status; git add f1.txt f2.txt f3.txt; git status; git commit -m "first commit"; git status; rm f3.txt; git status; git add f3.txt; git status; git commit -m "second commit"; git status; git restore --source=hash1 . check WD git status git add f3.txt git status git restore --source=hash2 .	 - f3.txt file is there as new one - Now it shows 1 things : 1. Untracked files: f3.txt, f4.txt - It adds the f3.txt (new file) into the index/staging area - Now it shows 2 things : 1. Changes to be committed: new file: f3.txt 2. Untracked files: f4.txt

check WD git status	- Since hash2 doesn't contain f3.txt, restoring from hash2 will make the f3.txt file as deleted bcz in this scenario f3.txt file was tracked - Now it shows : 1. Changes to be committed: new file: f3.txt 2. Changes not staged for commit: deleted: f3.txt 3. Untracked files: f4.txt